

BABU BANARASI DAS UNIVERSITY, LUCKNOW



NoSQL DATABASE

(Subject Code: BCADSN23202)

LAB FILE

SUBMITTED TO:
Mr. Harendra Singh

SUBMITTED BY:
Name: Harshit Kumar Srivastava
Roll No: 1250258215
Class: DS23

INDEX

Sr. No	Name of Experiment	Date	Faculty Signature	Remarks
1	Perform CRUD (Create, Read, Update, Delete) operations on a dataset using MongoDB			
2				
3				
4				
5				
6				
7				
8				
9				
10				

Practical: 1

Definition: MongoDB is a NoSQL, document-oriented database that stores data in flexible, JSON-like documents (BSON) without a fixed schema. CRUD stands for Create (insert), Read (find), Update and Delete, which are the four basic operations performed on any database.

Outcomes/Learning: Understood the structure of collections and documents in MongoDB and learned to perform insert, find, update and delete operations using query operators and filters.

Required Tool: MongoDB Community Server, MongoDB Shell (mongosh) and MongoDB Compass.

Working: A database 'college' and a collection 'students' were created and 22 student records were inserted. CRUD operations were then performed step by step using mongosh.

Step 1: Create database and collection

```
use college
db.createCollection("students")
```

Output:

```
switched to db college
{ ok: 1 }
```

Database 'college' and collection 'students' were created successfully.

Step 2: Insert multiple documents (Create)

```
db.students.insertMany([
  { roll_no: 1, name: "Harshit Srivastava", department: "CSE",
    semester: 5, marks: 82, city: "Lucknow" },
  { roll_no: 2, name: "Devansh Singh", department: "IT",
    semester: 3, marks: 74, city: "Kanpur" },
  { roll_no: 3, name: "Gauri Awasthi", department: "CSE",
    semester: 5, marks: 88, city: "Lucknow" },
  { roll_no: 4, name: "Harshita Singh", department: "IT",
    semester: 3, marks: 67, city: "Varanasi" },
  { roll_no: 5, name: "Ayushman Mishra", department: "ECE",
    semester: 1, marks: 91, city: "Prayagraj" },
  { roll_no: 6, name: "Prathana Bajaj", department: "CSE",
    semester: 5, marks: 78, city: "Lucknow" },
  { roll_no: 7, name: "Shrishti Singh", department: "IT",
    semester: 3, marks: 85, city: "Kanpur" },
  { roll_no: 8, name: "Rhimsha Khan", department: "CSE",
    semester: 5, marks: 62, city: "Lucknow" },
  { roll_no: 9, name: "Anchal Gupta", department: "ECE",
    semester: 1, marks: 89, city: "Agra" },
  { roll_no: 10, name: "Hetish Singh", department: "IT",
    semester: 3, marks: 55, city: "Noida" },
  { roll_no: 11, name: "Aditya Verma", department: "ME",
    semester: 7, marks: 58, city: "Gorakhpur" },
  { roll_no: 12, name: "Riya Sharma", department: "CSE",
    semester: 5, marks: 93, city: "Lucknow" },
  { roll_no: 13, name: "Kunal Yadav", department: "IT",
    semester: 3, marks: 47, city: "Ayodhya" },
  { roll_no: 14, name: "Saumya Tiwari", department: "ECE",
    semester: 1, marks: 71, city: "Kanpur" },
  { roll_no: 15, name: "Aman Pandey", department: "ME",
```

```

    semester: 7, marks: 66, city: "Varanasi" },
    { roll_no: 16, name: "Neha Rastogi", department: "CSE",
      semester: 5, marks: 80, city: "Bareilly" },
    { roll_no: 17, name: "Vivek Dubey", department: "IT",
      semester: 3, marks: 59, city: "Lucknow" },
    { roll_no: 18, name: "Priya Kushwaha", department: "ECE",
      semester: 1, marks: 76, city: "Agra" },
    { roll_no: 19, name: "Mohit Chauhan", department: "ME",
      semester: 7, marks: 52, city: "Noida" },
    { roll_no: 20, name: "Ananya Srivastava", department: "CSE",
      semester: 5, marks: 95, city: "Lucknow" },
    { roll_no: 22, name: "Khushi Gupta", department: "ECE",
      semester: 1, marks: 84, city: "Kanpur" }
  ])

```

Output:

```

{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('...'),
    '1': ObjectId('...'),
    ...
    '20': ObjectId('...')
  }
}

```

21 student documents were inserted into the collection using insertMany().

Step 3: Insert a single document and count

```

db.students.insertOne(
  { roll_no: 21, name: "Shivam Mishra", department: "IT",
    semester: 3, marks: 69, city: "Prayagraj" }
)

db.students.countDocuments()

```

Output:

```

{ acknowledged: true, insertedId: ObjectId('...') }
22

```

One more document was added with insertOne(); the collection now has 22 documents.

Step 4: View all documents (Read)

```
db.students.find().pretty()
```

Output:

Roll	Name	Dept	Sem	Marks	City
1	Harshit Srivastava	CSE	5	82	Lucknow
2	Devansh Singh	IT	3	74	Kanpur
3	Gauri Awasthi	CSE	5	88	Lucknow
4	Harshita Singh	IT	3	67	Varanasi
5	Ayushman Mishra	ECE	1	91	Prayagraj
6	Prathana Bajaj	CSE	5	78	Lucknow
7	Shrishti Singh	IT	3	85	Kanpur
8	Rhimsha Khan	CSE	5	62	Lucknow
9	Anchal Gupta	ECE	1	89	Agra
10	Hetish Singh	IT	3	55	Noida
11	Aditya Verma	ME	7	58	Gorakhpur
12	Riya Sharma	CSE	5	93	Lucknow
13	Kunal Yadav	IT	3	47	Ayodhya
14	Saumya Tiwari	ECE	1	71	Kanpur
15	Aman Pandey	ME	7	66	Varanasi
16	Neha Rastogi	CSE	5	80	Bareilly
17	Vivek Dubey	IT	3	59	Lucknow
18	Priya Kushwaha	ECE	1	76	Agra
19	Mohit Chauhan	ME	7	52	Noida
20	Ananya Srivastava	CSE	5	95	Lucknow
21	Shivam Mishra	IT	3	69	Prayagraj
22	Khushi Gupta	ECE	1	84	Kanpur

All 22 student documents are displayed using the find() method.

Step 5: Find a single student

```
db.students.findOne({ name: "Gauri Awasthi" })
```

Output:

```
{
  _id: ObjectId('...'),
  roll_no: 3,
  name: 'Gauri Awasthi',
  department: 'CSE',
  semester: 5,
  marks: 88,
  city: 'Lucknow'
}
```

The document of the student Gauri Awasthi is returned by findOne().

Step 6: Find students with a condition

```
db.students.find({ department: "CSE" })
```

Output:

Roll	Name	Dept	Sem	Marks	City
1	Harshit Srivastava	CSE	5	82	Lucknow
3	Gauri Awasthi	CSE	5	88	Lucknow
6	Prathana Bajaj	CSE	5	78	Lucknow
8	Rhimsha Khan	CSE	5	62	Lucknow
12	Riya Sharma	CSE	5	93	Lucknow
16	Neha Rastogi	CSE	5	80	Bareilly
20	Ananya Srivastava	CSE	5	95	Lucknow

Only the 7 students of the CSE department are displayed.

Step 7: Use the comparison operator \$gt

```
db.students.find({ marks: { $gt: 80 } })
```

Output:

Roll	Name	Dept	Sem	Marks	City
1	Harshit Srivastava	CSE	5	82	Lucknow
3	Gauri Awasthi	CSE	5	88	Lucknow
5	Ayushman Mishra	ECE	1	91	Prayagraj
7	Shrishti Singh	IT	3	85	Kanpur
9	Anchal Gupta	ECE	1	89	Agra
12	Riya Sharma	CSE	5	93	Lucknow
20	Ananya Srivastava	CSE	5	95	Lucknow
22	Khushi Gupta	ECE	1	84	Kanpur

The 8 students who scored more than 80 marks are displayed using \$gt.

Step 8: Use multiple conditions (AND)

```
db.students.find({ department: "CSE", city: "Lucknow" })
```

Output:

Roll	Name	Dept	Sem	Marks	City
1	Harshit Srivastava	CSE	5	82	Lucknow
3	Gauri Awasthi	CSE	5	88	Lucknow
6	Prathana Bajaj	CSE	5	78	Lucknow
8	Rhimsha Khan	CSE	5	62	Lucknow
12	Riya Sharma	CSE	5	93	Lucknow
20	Ananya Srivastava	CSE	5	95	Lucknow

The 6 CSE students who live in Lucknow are displayed using two conditions.

Step 9: Use the \$in operator

```
db.students.find({ department: { $in: ["IT", "ECE"] } })
```

Output:

Roll	Name	Dept	Sem	Marks	City
2	Devansh Singh	IT	3	74	Kanpur
4	Harshita Singh	IT	3	67	Varanasi
5	Ayushman Mishra	ECE	1	91	Prayagraj
7	Shrishti Singh	IT	3	85	Kanpur
9	Anchal Gupta	ECE	1	89	Agra
10	Hetish Singh	IT	3	55	Noida
13	Kunal Yadav	IT	3	47	Ayodhya
14	Saumya Tiwari	ECE	1	71	Kanpur
17	Vivek Dubey	IT	3	59	Lucknow
18	Priya Kushwaha	ECE	1	76	Agra
21	Shivam Mishra	IT	3	69	Prayagraj
22	Khushi Gupta	ECE	1	84	Kanpur

The 12 students of the IT and ECE departments are displayed using \$in.

Step 10: Projection, sort and limit

```
db.students.find({}, { name: 1, marks: 1, _id: 0 })
.sort({ marks: -1 })
.limit(5)
```

Output:

Name	Marks
Ananya Srivastava	95
Riya Sharma	93
Ayushman Mishra	91
Anchal Gupta	89
Gauri Awasthi	88

Name and marks of the top 5 scorers are displayed using projection, sort() and limit().

Step 11: Update a single document (Update)

```
db.students.updateOne(
  { name: "Harshit Srivastava" },
  { $set: { marks: 86 } }
)
```

Output:

```
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
```

Marks of Harshit Srivastava were changed from 82 to 86 using \$set.

Step 12: Update using the \$inc operator

```
db.students.updateOne(
  { name: "Harshita Singh" },
  { $inc: { marks: 5 } }
)
```

Output:

```
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
```

5 grace marks were added to Harshita Singh (67 to 72) using \$inc.

Step 13: Update multiple documents (add a new field)

```
db.students.updateMany(
  { city: { $ne: "Lucknow" } },
  { $set: { hostel: true } }
)
```

Output:

```
{ acknowledged: true, matchedCount: 15, modifiedCount: 15 }
```

A new field 'hostel: true' was added for the 15 students living outside Lucknow.

Step 14: Update multiple documents (promote students)

```
db.students.updateMany(  
  { department: "IT", semester: 3 },  
  { $set: { semester: 4 } }  
)
```

Output:

```
{ acknowledged: true, matchedCount: 7, modifiedCount: 7 }
```

7 IT students of Semester 3 were promoted to Semester 4 using updateMany().

Step 15: Delete a single document (Delete)

```
db.students.deleteOne({ name: "Kunal Yadav" })
```

Output:

```
{ acknowledged: true, deletedCount: 1 }
```

The record of Kunal Yadav was deleted using deleteOne().

Step 16: Delete multiple documents

```
db.students.deleteMany({ marks: { $lt: 60 } })
```

Output:

```
{ acknowledged: true, deletedCount: 4 }
```

4 students who scored less than 60 marks were deleted using deleteMany().

Step 17: Verify the final collection

```
db.students.find().pretty()  
  
db.students.countDocuments()
```

Output:

Roll	Name	Dept	Sem	Marks	City	Hostel
1	Harshit Srivastava	CSE	5	86	Lucknow	-
2	Devansh Singh	IT	4	74	Kanpur	true
3	Gauri Awasthi	CSE	5	88	Lucknow	-
4	Harshita Singh	IT	4	72	Varanasi	true
5	Ayushman Mishra	ECE	1	91	Prayagraj	true
6	Prathana Bajaj	CSE	5	78	Lucknow	-
7	Shrishti Singh	IT	4	85	Kanpur	true
8	Rhimsha Khan	CSE	5	62	Lucknow	-
9	Anchal Gupta	ECE	1	89	Agra	true
12	Riya Sharma	CSE	5	93	Lucknow	-
14	Saumya Tiwari	ECE	1	71	Kanpur	true
15	Aman Pandey	ME	7	66	Varanasi	true
16	Neha Rastogi	CSE	5	80	Bareilly	true
18	Priya Kushwaha	ECE	1	76	Agra	true
20	Ananya Srivastava	CSE	5	95	Lucknow	-
21	Shivam Mishra	IT	4	69	Prayagraj	true
22	Khushi Gupta	ECE	1	84	Kanpur	true

17

17 documents remain in the collection after all update and delete operations.

Result: All the CRUD (Create, Read, Update, Delete) operations were successfully performed on the 'students' dataset using MongoDB, and the outputs were verified at each step.